

NUnit Unit Testing Hands-On

Step 1 — Understand the Requirement

Login Function Behaviour — Three Execution Paths

```
Execution Path 1 - Valid credentials      Input:
userId="user_11",           password="secret@user11"
userId="user_22", password="secret@user22"      Output:
"Welcome user_11!!!" / "Welcome user_22!!!"
```

```
Execution Path 2 - Invalid credentials (wrong user or wrong password)
Input:  userId="user_11", password="wrongpass"      Output: "Invalid user
id/password"
```

```
Execution Path 3 - Empty / null input      Input:
userId="" or password="" (or null)
Output: throws ArgumentException
```

Valid credentials table: user_11 | secret@user11 user_22 | secret@user22

Step 2 — Source Project: AccountsManagerLib

Project / Solution Structure

```
Solution: AccountsManagerLib.sln
|  +-- AccountsManagerLib/           (Class Library - source
project) |  AccountsManager.cs      |  UserCredentials.cs
|  +-- AccountsManagerLib.Tests/     (Class Library - test
project) |  AccountsManagerTests.cs
```

UserCredentials.cs — Credential Model

```
namespace AccountsManagerLib
{
    public class UserCredentials
    {
        public string UserId { get; set; }
        public string Password { get; set; }
    }
}
```

AccountsManager.cs — Class Under Test

```
using System;
using System.Collections.Generic;

namespace AccountsManagerLib
{
    public class AccountsManager
    {
        // Valid credentials
        store
        private readonly List<UserCredentials> _validUsers =
        new List<UserCredentials>
        {
            new UserCredentials { UserId = "user_11", Password = "secret@user11" },
            new UserCredentials { UserId = "user_22", Password = "secret@user22" }
        };

        /// <summary>          /// Authenticates the user against the
        valid credentials list.

        /// Path 1: valid    → "Welcome <user_id>!!!"
        /// Path 2: invalid → "Invalid user id/password"
        /// Path 3: empty    → throws ArgumentException
        /// </summary>
```

```

        public string Login(string userId, string password)
        {
            // Execution Path 3 – missing input
            if (string.IsNullOrEmpty(userId))
                throw new ArgumentException("User id cannot be empty.");

            if (string.IsNullOrEmpty(password))
                throw new ArgumentException("Password cannot be empty.");

            // Execution Path 1 – valid credentials
            foreach (var user in _validUsers)
            {
                if (user.UserId == userId && user.Password == password)
                    return $"Welcome {userId}!!!";
            }

            // Execution Path 2 – invalid credentials
            return "Invalid user id/password";
        }
    }

    Build output (AccountsManagerLib):
    Build succeeded.
    0 Warning(s)
    0 Error(s)

```

Step 3 — Create and Configure the Test Project: AccountsManagerLib.Tests

Project Setup in Visual Studio

1. Right-click on Solution → Add → New Project
2. Choose: Class Library (.NET)
3. Name: AccountsManagerLib.Tests
4. Rename Class1.cs → AccountsManagerTests.cs

Install NuGet Packages

```

# Select AccountsManagerLib.Tests in Package Manager Console

Install-Package NUnit
Install-Package NUnit3TestAdapter
Install-Package Microsoft.NET.Test.Sdk

Successfully installed 'NUnit 3.14.0' to AccountsManagerLib.Tests
Successfully installed 'NUnit3TestAdapter 4.5.0' to AccountsManagerLib.Tests
Successfully installed 'Microsoft.NET.Test.Sdk' to AccountsManagerLib.Tests

```

Add Project Reference to AccountsManagerLib

```

# Right-click AccountsManagerLib.Tests
# → Add → Project Reference → tick AccountsManagerLib → OK
Reference added: AccountsManagerLib.Tests → AccountsManagerLib ✓

```

Step 4 — Write the Test Class: AccountsManagerTests.cs

Naming Conventions

Test Project : AccountsManagerLib.Tests Test Class : AccountsManagerTests (SUT = AccountsManager) Test Methods :
 UnitUnderTest_Scenario_ExpectedOutcome Rule : One Assert.That() per test method (Single Assertion Rule)

AccountsManagerTests.cs — Full Test Class

```

using NUnit.Framework;
using AccountsManagerLib;
using System;

namespace AccountsManagerLib.Tests
{
    [TestFixture]

```

```

    public class AccountsManagerTests
    {
        private AccountsManager _accountsManager;

        [SetUp]
        public void SetUp()
        {
            // Fresh instance created before every single test
            _accountsManager = new AccountsManager();
        }

        //
        // Execution Path 1 - Valid Credentials
        //

        [Test]
        public void Login_ValidCredentialsUser11_ReturnsWelcomeMessage()
        {
            // Arrange
            string userId = "user_11";
            string password = "secret@user11";

            // Act
            string result = _accountsManager.Login(userId, password);

            // Assert - Single Assertion Rule
            Assert.That(result, Is.EqualTo("Welcome user_11!!!"));
        }

        [Test]
        public void Login_ValidCredentialsUser22_ReturnsWelcomeMessage()
        {
            string userId = "user_22";
            string password = "secret@user22";

            string result = _accountsManager.Login(userId, password);

            Assert.That(result, Is.EqualTo("Welcome user_22!!!"));
        }

        [Test]
        public void Login_ValidCredentialsUser11_WelcomeMessageContainsUserId()
        {
            string result = _accountsManager.Login("user_11", "secret@user11");

            // Welcome message must include the user id
            Assert.That(result, Does.Contain("user_11"));
        }

        [Test]
        public void Login_ValidCredentials_WelcomeMessageContainsExclamationMarks()
        {
            string result = _accountsManager.Login("user_11", "secret@user11");

            Assert.That(result, Does.Contain("!!!"));
        }

        [Test]
        public void Login_ValidCredentials_ResultIsNotNull()
        {
            string result = _accountsManager.Login("user_22", "secret@user22");

            Assert.That(result, Is.Not.Null);
        }

        //
        // Execution Path 2 - Invalid Credentials
        //

        [Test]

```

```

        public void Login_WrongPassword_ReturnsInvalidMessage()
        {
            // Arrange
            string userId = "user_11";
            string password = "wrongpassword";

            // Act
            string result = _accountsManager.Login(userId, password);

            // Assert
            Assert.That(result, Is.EqualTo("Invalid user id/password"));
        }

        [Test]
        public void Login_WrongUserId_ReturnsInvalidMessage()
        {
            string result = _accountsManager.Login("unknown_user", "secret@user11");

            Assert.That(result, Is.EqualTo("Invalid user id/password"));
        }

        [Test]
        public void Login_BothWrongUserIdAndPassword_ReturnsInvalidMessage()
        {
            string result = _accountsManager.Login("hacker", "badpass");

            Assert.That(result, Is.EqualTo("Invalid user id/password"));
        }

        [Test]
        public void Login_CorrectUserIdButWrongCase_ReturnsInvalidMessage()
        {
            // Credentials are case-sensitive
            string result = _accountsManager.Login("USER_11", "secret@user11");

            Assert.That(result, Is.EqualTo("Invalid user id/password"));
        }

        [Test]
        public void Login_CorrectPasswordButWrongCase_ReturnsInvalidMessage()
        {
            string result = _accountsManager.Login("user_11", "SECRET@USER11");
            Assert.That(result, Is.EqualTo("Invalid user id/password"));
        }

        [Test]
        public void Login_SwappedCredentials_ReturnsInvalidMessage()
        {
            // user_11 with user_22's password - should still be invalid
            string result = _accountsManager.Login("user_11", "secret@user22");

            Assert.That(result, Is.EqualTo("Invalid user id/password"));
        }

        [Test]
        public void Login_InvalidCredentials_ResultDoesNotContainWelcome()
        {
            string result = _accountsManager.Login("bad_user", "bad_pass");

            Assert.That(result, Does.Not.Contain("Welcome"));
        }

        //
        // Execution Path 3 - Empty / Null Input → ArgumentException
        //

        [Test]
        public void Login_EmptyUserId_ThrowsArgumentException()
        {
            // Assert - verifies that ArgumentException is
            thrown
            Assert.That(
                () =>
                _accountsManager.Login("", "secret@user11"),
                Throws.TypeOf<ArgumentException>());
        }

```

```

[Test]
public void Login_EmptyPassword_ThrowsArgumentException()
{
    Assert.That(
        () => _accountsManager.Login("user_11", ""),
        Throws.TypeOf<ArgumentException>());
}

[Test]
public void Login_NullUserId_ThrowsArgumentException()
{
    Assert.That(
        () =>
        _accountsManager.Login(null, "secret@user11"),
        Throws.TypeOf<ArgumentException>());
}

[Test]
public void Login_NullPassword_ThrowsArgumentException()
{
    Assert.That(
        () => _accountsManager.Login("user_11", null),
        Throws.TypeOf<ArgumentException>());
}

[Test]
public void Login_WhitespaceUserId_ThrowsArgumentException()
{
    Assert.That(
        () => _accountsManager.Login(
        " ", "secret@user11"),
        Throws.TypeOf<ArgumentException>());
}

[Test]
public void Login_WhitespacePassword_ThrowsArgumentException()
{
    Assert.That(
        () => _accountsManager.Login("user_11", " "),
        Throws.TypeOf<ArgumentException>());
}

[Test]
public void Login_BothFieldsEmpty_ThrowsArgumentException()
{
    Assert.That(
        () => _accountsManager.Login("", ""),
        Throws.TypeOf<ArgumentException>());
}
}

```

Step 5 — Run Tests (All Pass)

Run from Visual Studio or CLI

```

# Visual Studio: Test → Run All Tests    (Ctrl + R, A)
# OR dotnet
test

```

Test Explorer Output — All Green

```

Test Run started... -----
Test Explorer - AccountsManagerLib.Tests

Path 1 - Valid Credentials
✓ Login_ValidCredentialsUser11_ReturnsWelcomeMessage ✓
  Login_ValidCredentialsUser22_ReturnsWelcomeMessage
✓ Login_ValidCredentialsUser11_WelcomeMessageContainsUserId
✓ Login_ValidCredentials_WelcomeMessageContainsExclamationMarks
✓ Login_ValidCredentials_ResultIsNotNull

Path 2 - Invalid Credentials

```

```

✓ Login_WrongPassword_ReturnsInvalidMessage
✓ Login_WrongUserId_ReturnsInvalidMessage
✓ Login_BothWrongUserIdAndPassword_ReturnsInvalidMessage
✓ Login_CorrectUserIdButWrongCase_ReturnsInvalidMessage
✓ Login_CorrectPasswordButWrongCase_ReturnsInvalidMessage
✓ Login_SwappedCredentials_ReturnsInvalidMessage
✓ Login_InvalidCredentials_ResultDoesNotContainWelcome

```

Path 3 – Empty / Null Input

```

✓ Login_EmptyUserId_ThrowsArgumentException
✓ Login_EmptyPassword_ThrowsArgumentException
✓ Login_NullUserId_ThrowsArgumentException
✓ Login_NullPassword_ThrowsArgumentException
✓ Login_WhitespaceUserId_ThrowsArgumentException
✓ Login_WhitespacePassword_ThrowsArgumentException
✓ Login_BothFieldsEmpty_ThrowsArgumentException

```

```

-----
Test Run Successful. Total tests: 19   Passed: 19   Failed: 0
Skipped: 0
Duration: 0.61 s

```

Step 6 — Break the Source Code and Rerun Tests

Defect 1 — Change Welcome Message Format in AccountsManager.cs

Remove the exclamation marks from the return string to simulate a typo.

```

// BROKEN – wrong welcome message format
return $"Welcome {userId}";      // missing "!!!" at the end Test

```

Explorer Output — Failures Detected

```

Test Run started...
-----
X Login_ValidCredentialsUser11_ReturnsWelcomeMessage
  Expected: "Welcome user_11!!!"
  But was:  "Welcome user_11"

X Login_ValidCredentialsUser22_ReturnsWelcomeMessage
  Expected: "Welcome user_22!!!"
  But was:  "Welcome user_22"

X Login_ValidCredentials_WelcomeMessageContainsExclamationMarks
  Expected: string containing "!!!"
  But was:  "Welcome user_11"

✓ Login_ValidCredentialsUser11_WelcomeMessageContainsUserId
✓ Login_ValidCredentials_ResultIsNotNull
✓ Login_WrongPassword_ReturnsInvalidMessage
✓ Login_WrongUserId_ReturnsInvalidMessage
... (all Path 2 and Path 3 tests still pass)

-----
Test Run Failed. Total tests: 19   Passed: 16   Failed: 3
Skipped: 0
Duration: 0.58 s

```

Defect 2 — Remove ArgumentException for Empty Input

Comment out the empty-check block to simulate a second defect.

```

// BROKEN – validation guard removed

```

```
// if (string.IsNullOrEmpty(userId)) //      throw new
ArgumentException("User id cannot be empty."); // if
(string.IsNullOrEmpty(password))

//      throw new ArgumentException("Password cannot be empty."); Test
```

Explorer Output — More Failures

```
Test Run started... -----
-----
```

```
X Login_EmptyUserId_ThrowsArgumentException
    Expected: <System.ArgumentException> to be thrown
    But was:  "Invalid user id/password" returned instead

X Login_EmptyPassword_ThrowsArgumentException
    Expected: <System.ArgumentException> to be thrown
    But was:  "Invalid user id/password" returned instead

X Login_NullUserId_ThrowsArgumentException
    Expected: <System.ArgumentException> to be thrown
    But was:  NullReferenceException thrown (uncaught)

X Login_NullPassword_ThrowsArgumentException
    Expected: <System.ArgumentException> to be thrown
    But was:  NullReferenceException thrown (uncaught)

X Login_WhitespaceUserId_ThrowsArgumentException
    Expected: <System.ArgumentException> to be thrown
    But was:  "Invalid user id/password" returned instead

X Login_WhitespacePassword_ThrowsArgumentException
    Expected: <System.ArgumentException> to be thrown
    But was:  "Invalid user id/password" returned instead

X Login_BothFieldsEmpty_ThrowsArgumentException
    Expected: <System.ArgumentException> to be thrown
    But was:  "Invalid user id/password" returned instead
```

```
-----
Test Run Failed. Total tests: 19   Passed: 12   Failed: 7
Skipped: 0
Duration: 0.63 s
```

Tests correctly caught both defects introduced in the source code.

Both defects were caught immediately by the tests: Defect 1 (wrong message format) → 3 tests failed Defect 2 (missing validation) → 7 tests failed Without tests, both bugs would have reached production undetected.

Step 7 — Restore Source Code and Final Run

Restore AccountsManager.cs to Correct Implementation

```
// Restore Path 1 return
return $"Welcome {userId}!!!";

// Restore Path 3 guards
if (string.IsNullOrEmpty(userId))
    throw new ArgumentException("User id cannot be empty."); if
(string.IsNullOrEmpty(password))

    throw new ArgumentException("Password cannot be empty."); Final
```

Test Run — All Green Again

```
Test Run started... -----
-----
✓ Login_ValidCredentialsUser11_ReturnsWelcomeMessage ✓
    Login_ValidCredentialsUser22_ReturnsWelcomeMessage
```

```

✓ Login_ValidCredentialsUser11_WelcomeMessageContainsUserId
✓ Login_ValidCredentials_WelcomeMessageContainsExclamationMarks
✓ Login_ValidCredentials_ResultIsNotNull
✓ Login_WrongPassword_ReturnsInvalidMessage
✓ Login_WrongUserId_ReturnsInvalidMessage
✓ Login_BothWrongUserIdAndPassword_ReturnsInvalidMessage
✓ Login_CorrectUserIdButWrongCase_ReturnsInvalidMessage
✓ Login_CorrectPasswordButWrongCase_ReturnsInvalidMessage
✓ Login_SwappedCredentials_ReturnsInvalidMessage
✓ Login_InvalidCredentials_ResultDoesNotContainWelcome
✓ Login_EmptyUserId_ThrowsArgumentException
✓ Login_EmptyPassword_ThrowsArgumentException
✓ Login_NullUserId_ThrowsArgumentException
✓ Login_NullPassword_ThrowsArgumentException
✓ Login_WhitespaceUserId_ThrowsArgumentException
✓ Login_WhitespacePassword_ThrowsArgumentException
✓ Login_BothFieldsEmpty_ThrowsArgumentException
-----

```

```

Test Run Successful. Total tests: 19   Passed: 19   Failed: 0
Skipped: 0
Duration: 0.59 s

```

NUnit Attributes and Assert.That() Patterns — Quick Reference

```

// Attributes
[TestFixture] // marks class as a test container [SetUp]           // runs
before every [Test] - creates fresh AccountsManager [Test]       // marks
each individual test method
// Assert.That() patterns used in this exercise
Assert.That(result, Is.EqualTo("Welcome user_11!!!"));           // exact match
Assert.That(result, Does.Contain("user_11"));                    // substring check
Assert.That(result, Does.Contain("!!!"));                         // contains exclamation
Assert.That(result, Does.Not.Contain("Welcome"));                 // negative substring
Assert.That(result, Is.NotNull);                                  // null check

// Exception testing pattern
Assert.That(
    () => _accountsManager.Login("", "pass"),
    Throws.TypeOf<ArgumentException>());                        // exception type check

```

Single Assertion Rule applied: each test method has exactly one `Assert.That()`. This ensures that when a test fails, the failure message immediately tells you which specific behaviour is broken.